

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage:40%

Dr

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze

7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

Code of Conduct:

I Sandra Sudevan(192424422) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sandra Sudevan

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

		satisfying all constraints			
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application ; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Heuristic Optimization of a Complex SQL Query

Aim

To optimize a complex SQL query using heuristic optimization rules such as **selection pushdown, projection pushdown, and join ordering**.

Given Query

```
SELECT S.Name, C.CourseName
FROM Student S
JOIN Enrollment E ON S.ID = E.StudentID
JOIN Course C ON E.CourseID = C.ID
WHERE S.Department = 'CSE'
AND C.Credits > 3;
```

Procedure

Step 1: Convert the query into relational algebra

$$\begin{array}{c} \pi \text{ Name, CourseName} \\ | \\ \sigma \text{ Department='CSE' AND Credits}>3 \\ | \\ S \bowtie E \bowtie C \end{array}$$

Step 2: Push selection down

$$S1 = \sigma \text{ Department='CSE'}(\text{Student})$$

$$C1 = \sigma \text{ Credits}>3(\text{Course})$$

This reduces the number of tuples before joining.

Step 3: Push projection down

$S2 = \pi_{ID, Name}(S1)$

$E2 = \pi_{StudentID, CourseID}(Enrollment)$

$C2 = \pi_{ID, CourseName}(C1)$

Only required attributes are retained.

Step 4: Order the joins

Join the smaller intermediate relations first:

$S2 \bowtie E2$
|
 $\bowtie C2$

Optimized Query Plan

$\pi_{Name, CourseName}$
|
JOIN
/ \
JOIN C2
/ \ |
S2 E2 $\sigma_{Credits > 3}$
| | |
 $\sigma_{Department = CSE}$ Course
|
Student

Result

The query is optimized by:

- Pushing selections down.
- Performing projections early.
- Joining smaller intermediate relations first.

This reduces the **intermediate relation size and query execution cost**.

2. Cost-Based Optimization of Join Algorithms

Aim

To select the most efficient join algorithm using **cost-based optimization**.

Given

Relation	Tuples	Pages
Student	1000	100
Course	100	10

Available memory = **20 pages**

Procedure

Step 1: Nested-Loop Join

Formula:

$$\text{Cost} = B(R) + B(R) \times B(S)$$

For Student $\bowtie$ Course:

$$\begin{aligned} &= 100 + (100 \times 10) \\ &= 1100 \text{ I/O} \end{aligned}$$

Step 2: Sort-Merge Join

Approximate formula:

$$\begin{aligned} \text{Cost} &= 2B(R) + 2B(S) \\ &= 2(100) + 2(10) \\ &= 220 \text{ I/O} \end{aligned}$$

Step 3: Hash Join

Approximate formula:

$$\begin{aligned} \text{Cost} &= 3(B(R) + B(S)) \\ &= 3(100 + 10) \\ &= 330 \text{ I/O} \end{aligned}$$

Comparison

Join Method	Estimated Cost
Nested-Loop Join	1100 I/O
Sort-Merge Join	220 I/O
Hash Join	330 I/O

Result

Sort-Merge Join is selected because it has the lowest estimated cost of **220 I/O operations** under these simplified assumptions.

3. JDBC Connectivity Using PreparedStatement

Aim

To establish JDBC connectivity between Java and MySQL and execute a parameterized query using PreparedStatement.

Requirements

- Java
- MySQL
- MySQL JDBC Driver
- MySQL database named college

Procedure

1. Load the MySQL JDBC driver.
2. Establish database connection.
3. Create a PreparedStatement.
4. Pass the required parameter.
5. Execute the query.
6. Display the result.
7. Close the connection.

Program

```
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/college";
        String user = "root";
        String password = "root";

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con =
                DriverManager.getConnection(url, user, password);
```

```

String sql = "SELECT * FROM Student WHERE Mark > ?";

PreparedStatement ps = con.prepareStatement(sql);

ps.setInt(1, 80);

ResultSet rs = ps.executeQuery();

while (rs.next()) {
    System.out.println(
        rs.getInt("ID") + " " +
        rs.getString("Name") + " " +
        rs.getInt("Mark")
    );
}

rs.close();
ps.close();
con.close();

} catch (Exception e) {
    System.out.println(e);
}
}
}

```

Result

The Java program successfully establishes a connection with MySQL and retrieves students whose marks are greater than **80** using PreparedStatement.

4. Conflict Serializability Using Precedence Graph

Aim

To determine whether a given transaction schedule is **conflict-serializable** using a precedence graph.

Given Schedule

```

T1: R(A) W(A) R(B)
T2:   R(A) W(B)
T3:   R(B) W(A)

```

Procedure

Step 1: Create transaction nodes

T1, T2, T3

Step 2: Identify conflicts

Conflicts occur when:

- Operations belong to different transactions.
- They access the same data item.
- At least one operation is a write.

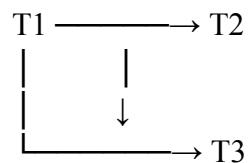
Edges obtained:

T1 → T2

T1 → T3

T2 → T3

Step 3: Draw precedence graph



Step 4: Check for cycle

There is **no cycle**.

Result

The schedule is **conflict-serializable**.

Equivalent serial order:

T1 → T2 → T3

5. Basic Two-Phase Locking and Deadlock

Aim

To demonstrate **Basic Two-Phase Locking (2PL)** and identify and resolve deadlocks.

Given

T1: Lock-X(A)

Read(A)

Lock-X(B)

T2: Lock-X(B)

Read(B)

Lock-X(A)

Procedure

Initially:

T1 holds A

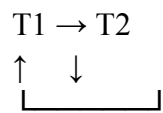
T2 holds B

Then:

T1 requests B → waits for T2

T2 requests A → waits for T1

Wait-For Graph



A cycle exists.

Therefore, **deadlock occurs**.

Deadlock Resolution

Abort T2:

ABORT T2

Release B

Now T1 obtains B:

T1 → Lock-X(B)

T1 → Complete

T1 → COMMIT

T2 can then be restarted.

2PL Phases

Growing Phase:

LOCK-X(A)

LOCK-X(B)

Shrinking Phase:

UNLOCK(A)

UNLOCK(B)

Result

The deadlock is detected using the wait-for graph and resolved by aborting **T2**.

6. Wait-Die and Wound-Wait Schemes

Aim

To apply **Wait-Die** and **Wound-Wait** timestamp-based deadlock prevention techniques.

Given

T1 timestamp = 10

T2 timestamp = 20

Therefore:

T1 = Older

T2 = Younger

Assume T2 holds lock X and T1 requests X.

Wait-Die

Older transaction requests a lock held by younger:

T1 requests X

T2 holds X

Since T1 is older:

T1 waits

Now suppose T2 requests a lock held by T1:

T2 requests X

T1 holds X

Since T2 is younger:

T2 aborts

Wound-Wait

If T1 requests a lock held by T2:

T1 requests X

T2 holds X

T1 is older, so:

T2 is aborted

T1 gets X

If T2 requests a lock held by T1:

T2 requests X

T1 holds X

T2 is younger:

T2 waits

Result

Scheme	Older → Younger	Younger → Older
Wait-Die	Wait	Abort
Wound-Wait	Abort younger	Wait

Both schemes prevent deadlocks by controlling transaction waiting.

7. Immediate Update Recovery

Aim

To recover a database using the **Immediate Update recovery protocol** after a system failure.

Given Log

<T1 START>

<T1, A, 100, 150>

<T2 START>

<T2, B, 200, 250>

<CHECKPOINT>

<T1 COMMIT>

<T2, C, 300, 350>

SYSTEM CRASH

Procedure

In Immediate Update:

- Database changes may be written before commit.
- **Committed transactions → REDO**
- **Uncommitted transactions → UNDO**

After the crash:

T1 → COMMITTED

T2 → UNCOMMITTED

Therefore:

REDO T1

UNDO T2

Recovery

T1:

A = 150

T2:

B = 200

C = 300

T2's changes are undone.

Result

The database contains all changes made by committed transaction **T1**, while changes made by uncommitted transaction **T2** are removed.

8. Deferred Update Recovery

Aim

To recover a database using the **Deferred Update (NO-UNDO/REDO)** technique.

Given Log

<T1 START>

<T1, A, 100, 150>

<T2 START>

<T2, B, 200, 250>

<T1 COMMIT>

<T2, C, 300, 350>

SYSTEM CRASH

Procedure

In Deferred Update:

- Changes are not written to the database until commit.
- Committed transactions are **REDO**.
- Uncommitted transactions are ignored.

After crash:

T1 → COMMITTED

T2 → UNCOMMITTED

Therefore:

REDO(T1)

IGNORE(T2)

Recovery

A = 150

T2's changes are not applied.

Result

Only the changes of committed transaction **T1** are reflected in the database.

9. Strict 2PL for Banking System

Aim

To design a concurrency control mechanism using **Strict Two-Phase Locking** for a banking transaction.

Given

Initial balances:

Account A = ₹1000

Account B = ₹500

T1 transfers ₹100 from A to B.

T2 simultaneously reads A's balance.

Procedure

Transaction T1

LOCK-X(A)

LOCK-X(B)

READ(A)

$A = A - 100$

READ(B)

$B = B + 100$

WRITE(A)

WRITE(B)

COMMIT

UNLOCK(A)

UNLOCK(B)

New balances:

A = ₹900

B = ₹600

Transaction T2

T2 tries to read A:

LOCK-S(A)

But T1 already holds:

LOCK-X(A)

Therefore:

T2 → WAIT

After T1 commits:

T1 → UNLOCK(A)

T2 → LOCK-S(A)

T2 → READ(A)

T2 reads:

A = ₹900

Result

Strict 2PL ensures that T2 does not read an intermediate balance. The final balances are:

Account A = ₹900

Account B = ₹600

Thus, **database consistency is maintained.**

10. Pseudocode for Heuristic Query Optimization

Aim

To develop a pseudocode algorithm for heuristic query optimization.

Procedure / Algorithm

Algorithm Heuristic_Optimization(Query)

BEGIN

 Parse the SQL Query

 Convert SQL Query into Relational Algebra

 // Selection Pushdown

 FOR each selection condition

 Move selection close to the base relation

END FOR

// Projection Pushdown

FOR each relation

 Remove unnecessary attributes

 Keep only required attributes

END FOR

// Join Ordering

Estimate the size of each relation

Select the smallest relation

Join the smallest relation first

Estimate intermediate result size

Continue joining the relation
that produces the smallest result

Remove unnecessary operations

Generate the optimized query plan

END

Result

The heuristic optimizer reduces query processing cost by:

Selection Pushdown



Projection Pushdown



Small Relation First



Small Intermediate Results



Optimized Query Plan